

ISM3232 — Week 6 Lab

Conditionals, Loops & Dictionaries — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 06 · Unit 2 · Python Foundations

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:03)	4
6.2 Part 1 — Records and Loop (0:03–0:15, 12 min)	4
6.3 Part 2 — Two Business Rules (0:15–0:29, 14 min)	6
6.4 Part 3 — Formatted Summary and File Output (0:29–0:45, 16 min)	8
6.5 Part 4 — pytest (0:45–0:58, 13 min)	11
6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)	14
6.7 Stretch — Sort and Group Output (1:06–1:15, as time allows)	15
7 Wrap-Up (last ~9 minutes)	16
8 Appendix A — Full Answer Key (week6_lab.py + tests/test_week6.py)	16
9 Appendix B — Extra Practice (only if the class finishes early)	18

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 6 Lab — Conditionals, Loops & Dictionaries
Unit	Unit 2 · Python Foundations
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full submission ritual
Prerequisites	Week 5: all four data types, operators, <code>input()</code> conversion, <code>pytest</code> basics (marked Midterm-Eligible)
Student-facing lab page	Week 6 In-Class Lab — Module 4, “Conditionals, Loops, Lists, and Dictionaries”
Parts covered	Part 1 (records + loop) – Part 5 (ritual + push) + Stretch (sort/group)
Submission	Git commit (message must include “lab 6”) + Canvas URL, completion credit

This is the lab where ISM3232 introduces the **list-of-dicts pattern** — the single most common way real business data actually gets represented in memory before it’s ever a database table or a pandas DataFrame: a list where every item is a dictionary with the same shape. Everything in this lab builds toward one genuinely realistic mini-application: loop through records, apply business rules with conditionals, accumulate results, and save a summary to disk. Verified this week: the lab page’s own provided `test_week6.py` passes cleanly as written — no hidden bugs this time, a useful contrast to worth naming for the room after last week’s diagnostic detour.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Represent a small dataset as a list of dictionaries, each with consistent keys, and loop through it with `for rec in records:`.
2. Apply `if` conditionals inside a loop to filter and classify records against business thresholds.
3. Accumulate multiple results simultaneously inside one loop — a running total and two separately-filtered lists.
4. Write a summary to a text file with `open(..., 'w')` and `.write()`, after ensuring the target folder exists with `os.makedirs(..., exist_ok=True)`.
5. Write `pytest` tests using list comprehensions and generator expressions as the assertion targets, not just simple hardcoded values.

3 Before Class — Setup Checklist

- ☐ Rehearse the full script once with your own five records before class — this guide uses the lab page’s own purchase-request template (Taylor/Jordan/Morgan/Riley/Alex) throughout for consistency; feel free to substitute your own domain live, but know your own numbers could either way.
- ☐ Confirm `data/` doesn’t already exist with conflicting content in your demo’s `module04_programming/` folder before running Part 3 live — `os.makedirs(..., exist_ok=True)` is safe to re-run, but a stale `week6_summary.txt` from a previous rehearsal can make live output confusing if you don’t cat it fresh.
- ☐ This week’s `test_week6.py`, unlike Week 5’s, has no hidden bugs when run as provided — worth stating that explicitly to the room as a point of contrast, especially if Week 5’s diagnostic detour was memorable; it reinforces that reading test output carefully is a permanent habit, not a one-time lesson tied to a single bad file.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module04_programming/` venv from Week 5
- Students: `week5_lab.py`’s project folder, continuing in the same venv/repo

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:03	Welcome: “the list-of-dicts pattern”	3
0:03–0:15	Part 1 — Records and loop	12
0:15–0:29	Part 2 — Two business rules	14
0:29–0:45	Part 3 — Formatted summary and file output	16
0:45–0:58	Part 4 — pytest	13
0:58–1:06	Part 5 — Ritual and push	8
1:06–1:15	Stretch (sort/group) + wrap-up	9

Four required parts plus the ritual fill the class period at a normal pace; the sort/group Stretch is genuinely quick (`sorted(..., key=lambda ...)`) and worth full live demo time if the room reaches it, since it’s a directly reusable pattern for later modules.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:03)

Say to the class:

“Today’s structure — a list where every item is a dictionary with the same fields — is genuinely one of the most common ways real business data lives in memory, before it’s ever in a spreadsheet, a database table, or a pandas DataFrame later in this course. Five records, two business rules, a formatted summary saved to disk. This is the first lab that feels like a small, real application, not just a syntax exercise.”

6.2 Part 1 — Records and Loop (0:03–0:15, 12 min)

Teaching goal: The list-of-dicts literal syntax, and a first loop that filters on one condition.

Say to the class:

“Five records, each a dictionary with the same five keys. This consistency — every record having exactly the same fields — is what makes looping through them meaningful.”

Live-code this:

```
cd ~/ism3232/module04_programming
source .venv/bin/activate
touch week6_lab.py && code week6_lab.py
```

```
# week6_lab.py
# Author: [Your Name]

records = [
    {'id': 1, 'name': 'Taylor', 'category': 'Travel', 'amount': 1200, 'status': 'Pending'},
    {'id': 2, 'name': 'Jordan', 'category': 'Equipment', 'amount': 450, 'status': 'Pending'},
    {'id': 3, 'name': 'Morgan', 'category': 'Software', 'amount': 3500, 'status': 'Approved'},
    {'id': 4, 'name': 'Riley', 'category': 'Travel', 'amount': 89, 'status': 'Pending'},
    {'id': 5, 'name': 'Alex', 'category': 'Equipment', 'amount': 2200, 'status': 'Pending'}
]

for rec in records:
    if rec['status'] == 'Pending':
        print(f"{rec['name']}: ${rec['amount']:,.2f}")
```

Line-by-line explanation:

- `records = [{...}, {...}, ...]` — a list, square brackets, five elements, each itself a dictionary in curly braces — say explicitly: every dictionary here has the **same five keys** (`id`, `name`, `category`, `amount`, `status`), in the same order, which is a convention worth calling out — nothing in Python *requires* every dict in a list to share the same keys, but for this pattern to be useful for looping and filtering, consistency is what makes it work.
- `for rec in records:` — `rec` is bound to one whole dictionary per pass, exactly the same loop shape as any prior list iteration, just holding a richer value each time than a plain number or string.
- `if rec['status'] == 'Pending':` — bracket-notation dictionary access, inside a conditional, inside a loop — the exact three-layer combination that’s been building since earlier in the semester, now doing real filtering work on business data for the first time.
- `f"{rec['name']}: ${rec['amount']:,.2f}"` — note the **single quotes inside, double quotes outside** the f-string — Python requires the inner and outer quote characters to differ (or requires escaping) so there’s no ambiguity about where the string ends; a good moment to flag if a student’s editor highlights this oddly.

Run it. Verified output (only the four Pending records; Morgan, Approved, is correctly excluded):

Taylor: \$1,200.00

Jordan: \$450.00

Riley: \$89.00

Alex: \$2,200.00

Common student mistakes to watch for:

- Mismatched key names between records (a typo like `'Statuss'` in one dict) — this doesn’t error; it just means that record’s `rec['status']` lookup would fail with `KeyError` the moment the loop reaches it — a good moment to have students proofread their own five records carefully before running, since a typo two records deep can be genuinely hard to spot by eye.
- Forgetting the `if` filter entirely and printing all five records — Morgan (Approved) would incorrectly appear; a good “does this output match what was asked” sanity check.

Check for understanding: “If a sixth record had `'status': 'pending'` (lowercase), would it be included by this filter?” (No — string comparison with `==` is case-sensitive; `'pending' == 'Pending'` is `False`. A good, concrete reminder of a lesson that recurs constantly whenever real-world data has inconsistent capitalization — worth flagging as a genuine, common data-quality issue, not just a syntax trivia point.)

6.3 Part 2 — Two Business Rules (0:15–0:29, 14 min)

Teaching goal: Accumulate multiple, independent results inside one loop — a running total, plus two separately-filtered lists built from two distinct threshold checks.

Say to the class:

“Same loop shape, but now doing three jobs at once: a running total, and two separate ‘flag this record’ lists, each with its own threshold. This is the accumulator pattern, applied three times in parallel, inside one pass through the data.”

Live-code this, replacing Part 1’s simple print loop:

```
LIMIT = 1000
HIGH = 2000
total = 0
flagged = []
high_value = []

for rec in records:
    if rec['status'] == 'Pending':
        total += rec['amount']
        if rec['amount'] > LIMIT:
            flagged.append(rec)
        if rec['amount'] > HIGH:
            high_value.append(rec)
```

Line-by-line explanation:

- `LIMIT = 1000`, `HIGH = 2000` — **named constants**, written in all-caps by convention (a widely-used style signal that a value is meant to stay fixed, not be reassigned elsewhere) — say explicitly why this matters: if the review threshold ever changes, there’s exactly one line to edit, rather than hunting through the script for every place 1000 appears literally.
- `total = 0`, `flagged = []`, `high_value = []` — **three separate accumulators, initialized before the loop**, each a different shape (a number, and two empty lists) for a different purpose — the same “initialize before, matching what you’re building” principle from every prior accumulator lesson this semester.
- `if rec['status'] == 'Pending':` — the outer filter, unchanged from Part 1 — say explicitly: **everything inside this if only runs for pending records**; approved records (like Morgan’s) never reach any of the three accumulator updates below, correctly excluding them from the total and both flagged lists.
- `total += rec['amount']` — the running total, only for pending records.
- `if rec['amount'] > LIMIT: flagged.append(rec)` — **note this appends the whole dictionary rec**, not just the amount — say explicitly why that matters: later parts of the script

(Part 3's "records needing review" printout) need the record's `id` and `name`, not just its dollar amount, so keeping the full dictionary is a deliberate choice, not an oversight.

- `if rec['amount'] > HIGH: high_value.append(rec)` — a **second, independent** check — note this is a separate `if`, not `elif`, since a record over `HIGH` (\$2000) is, by definition, also over `LIMIT` (\$1000) and should land in *both* lists, not just one. Ask the room explicitly: "would `elif` here be correct?" (No — `elif` would mean a high-value record only ever lands in `high_value`, never also in `flagged`, silently excluding it from the broader review list — a direct callback to earlier lessons on when independent `ifs` versus mutually-exclusive `elif` chains are the right structural choice.)

Run it and print the results to confirm. Verified values:

`total: 3939`

`flagged: ['Taylor', 'Alex']` # amounts over \$1,000: \$1,200 and \$2,200

`high_value: ['Alex']` # amounts over \$2,000: \$2,200 only

Common student mistakes to watch for:

- Using `elif` instead of two separate `ifs` for the `LIMIT`/`HIGH` checks — discussed above; this is the exercise's central structural point, worth catching explicitly if it happens.
- Placing `total += rec['amount']` **outside** the `if rec['status'] == 'Pending':` block by an indentation slip — would incorrectly include Morgan's \$3,500 Approved amount in the pending total; a good "does this total look right" check against the four pending amounts' actual sum ($1200 + 450 + 89 + 2200 = 3939$).
- Appending `rec['amount']` (just the number) instead of `rec` (the whole dictionary) to `flagged/high_value` — works fine for a total or count, but breaks Part 3's printout, which needs `r['id']` and `r['name']` from each flagged record — a good forward-looking reason to keep the full dict.

Check for understanding: "If a pending record had amount exactly equal to 1000 — right at `LIMIT` — would it be flagged?" (No — the check is strictly `> LIMIT`, not `>=`; a good boundary-condition check, echoing every prior module's repeated emphasis on reading comparison operators precisely.)

6.4 Part 3 — Formatted Summary and File Output (0:29–0:45, 16 min)

Teaching goal: Print a clean, multi-line summary, and — new today — **save it to an actual file on disk**, ensuring the target folder exists first.

Say to the class:

“Everything so far has only lived on screen, gone the moment the script ends. Today we save a summary to a real file — and I want to show you the one-line safety net that makes this work reliably regardless of whether the folder it’s going into already exists.”

Live-code this:

```
import os
os.makedirs('data', exist_ok=True)

lines = [
    f'Pending total:    ${total:,.2f}',
    f'Needs review:     {len(flagged)}',
    f'High-value:        {len(high_value)}',
]
for line in lines:
    print(line)

with open('data/week6_summary.txt', 'w') as f:
    f.write('\n'.join(lines) + '\n')

print('\nRecords needing review:')
for r in flagged:
    print(f"  ID {r['id']}: {r['name']} - ${r['amount']:,.2f}")
```

Line-by-line explanation:

- `import os` — Python’s built-in module for interacting with the operating system, including the filesystem — new today.
- `os.makedirs('data', exist_ok=True)` — **this is the line worth explaining most carefully.** Without it, `open('data/week6_summary.txt', 'w')` would fail with `FileNotFoundError` if the `data/` folder didn’t already exist — Python won’t create missing parent folders automatically just because you’re writing a file into one. `os.makedirs('data', ...)` creates the folder if it’s missing; **`exist_ok=True` is the specific detail that prevents an error if the folder is already there** — without it, re-running this script a second time (very likely, in the normal course of development and testing) would crash on the *second* run, even though nothing is actually wrong. Say explicitly: **this is worth using as a default habit any time a script writes to a folder that might or might not already exist.**

- `lines = [f'...', f'...', f'...']` — a list of three formatted strings, built up first, rather than printed and written separately — say explicitly why this structure is worth the extra step: building the content once, as a list, means both the `print()` loop and the file-write step below can reuse the *exact same* content, guaranteeing the screen output and the saved file never accidentally drift out of sync.
- `with open('data/week6_summary.txt', 'w') as f:` — the `with` block from earlier file-handling work, 'w' mode (create-or-overwrite, same overwrite-risk lesson as every prior file-writing exercise this semester).
- `f.write('\n'.join(lines) + '\n')` — `'\n'.join(lines)` is new syntax worth explaining precisely: `.join()` is a string method, called *on* the separator (here, `'\n'`, a newline character), that glues together every element of the list passed to it, inserting that separator *between* each pair of elements — say explicitly: this produces the three lines joined with newlines *between* them, but **not** a trailing newline after the last one, which is why the final `+ '\n'` is added explicitly, giving the file a clean trailing newline (a small but genuinely common convention for well-formed text files).
- The final loop over `flagged`, printing each review-needed record's `id`, `name`, and formatted amount — reusing Part 2's accumulated list directly.

Run it. Verified output:

```
Pending total:    $3,939.00
Needs review:     2
High-value:       1
```

```
Records needing review:
  ID 1: Taylor - $1,200.00
  ID 5: Alex - $2,200.00
```

Verify the file was written:

```
cat data/week6_summary.txt
```

Verified file content (note: matches the first three printed lines exactly, but *not* the “Records needing review” section — only `lines` was written to the file, by design):

```
Pending total:    $3,939.00
Needs review:     2
High-value:       1
```

Point out explicitly: the saved file is *shorter* than the full terminal output — only the three summary lines from `lines` were written, not the per-record review list printed afterward. Ask the room: “is this a bug, or a deliberate design choice?” (A defensible design choice, not a bug — the file captures a compact summary suitable for, say, a nightly log, while the terminal shows the fuller detail for whoever’s running the script interactively right now; but it’s worth explicitly noting that if the *file* were meant to include the review-needed detail too, that would require adding those lines to `lines` — or writing a second file — before the `with open(...)` block runs, since the file’s content is fixed

at the moment `.write()` executes.)

Common student mistakes to watch for:

- Skipping `os.makedirs(..., exist_ok=True)`, assuming `open(..., 'w')` will create the folder automatically — it will not; `FileNotFoundError` is the resulting error, worth demonstrating live if it doesn't happen naturally.
- Using `'\n'.join(lines)` without the trailing `+ '\n'` — the file's last line then has no trailing newline, which is usually harmless but can look visually odd in some tools/terminals (cat's prompt appearing directly attached to the last line rather than on its own line) — worth a quick before/after comparison if time allows.
- Confusing `f.write(...)` with `print(...)` — `.write()` does not add a newline automatically the way `print()` does, which is precisely why the `'\n'.join(...)` + `'\n'` construction is necessary here at all; a good moment to state this contrast explicitly, since it's a real, recurring source of “why did my file's lines all run together” confusion.

Check for understanding: “If you ran this script three times in a row without deleting `data/week6_summary.txt` between runs, what would the file contain after the third run?” (Still just the three summary lines — `'w'` mode overwrites completely on every run, not accumulating; a good, direct callback to earlier modules' `"w"`-vs-append-mode distinction, confirming it still applies here.)

6.5 Part 4 — pytest (0:45–0:58, 13 min)

Teaching goal: Five pytest tests using **list and generator comprehensions** as their core logic — genuinely new syntax, worth explaining carefully even though it's compact.

Say to the class:

“Five tests, and this time — unlike last week — they all pass exactly as written. What’s new here isn’t a bug to diagnose; it’s a genuinely useful piece of syntax: comprehensions, a compact way to build a filtered list in one line instead of a full `for/if/.append()` block.”

Live-code this:

touch tests/test_week6.py

code tests/test_week6.py

```
def test_accumulator():
    amounts = [500, 1500, 200]
    total = sum(a for a in amounts)
    assert total == 2200

def test_flag_filter():
    records = [{'amount': 500}, {'amount': 1500}, {'amount': 800}]
    flagged = [r for r in records if r['amount'] > 1000]
    assert len(flagged) == 1

def test_status_filter():
    records = [{'amount': 500, 'status': 'Pending'}, {'amount': 1000, 'status': 'Approved'}]
    pending = [r for r in records if r['status'] == 'Pending']
    assert len(pending) == 1

def test_dict_key_access():
    rec = {'id': 1, 'amount': 750, 'status': 'Pending'}
    assert rec['amount'] == 750

def test_list_of_dicts_length():
    data = [{'x': 1}, {'x': 2}, {'x': 3}]
    assert len(data) == 3
```

Line-by-line explanation:

- `sum(a for a in amounts)` — this is a **generator expression** inside `sum()`: `a for a in amounts` produces each value of `amounts` one at a time, and `sum()` adds them all up. Say explicitly: for this simple case, this is functionally identical to just `sum(amounts)` directly — the generator expression form is shown here specifically because it previews the more powerful

comprehension syntax on the next line, where filtering gets added.

- `[r for r in records if r['amount'] > 1000]` — a **list comprehension**: read it left to right as “for each `r` in `records`, if `r['amount'] > 1000`, include `r` in the new list.” Say explicitly this is a direct, compact equivalent to:

```
flagged = []
for r in records:
    if r['amount'] > 1000:
        flagged.append(r)
```

worth writing both versions side by side on the board once, so students see the comprehension isn't new *logic*, just a more compact *syntax* for exactly the accumulator-with-a-filter pattern from Part 2.

- `[r for r in records if r['status'] == 'Pending']` — the same comprehension shape, filtering on a different condition — reinforcing that the pattern generalizes to any filter, not just numeric thresholds.
- The final two tests (`test_dict_key_access`, `test_list_of_dicts_length`) — simpler, direct assertions confirming basic dictionary access and `len()` on a list of dicts, without comprehensions — worth noting these two are testing foundational operations the comprehension-based tests above depend on implicitly.

Run it:

```
pytest -v
```

Verified output — all five pass as provided, no diagnostic detour needed this week:

```
tests/test_week6.py::test_accumulator PASSED
tests/test_week6.py::test_flag_filter PASSED
tests/test_week6.py::test_status_filter PASSED
tests/test_week6.py::test_dict_key_access PASSED
tests/test_week6.py::test_list_of_dicts_length PASSED
5 passed
```

Say explicitly, as a brief, deliberate contrast to last week: “Notice I’m not diagnosing anything today — this file is correct as given. Last week’s lesson wasn’t ‘always expect broken tests’ — it was ‘read the output carefully regardless of what you expect.’ This week confirms that habit pays off either way: a quick, confident green run is just as much the result of careful reading as catching a real bug was.”

Common student mistakes to watch for:

- Trying to write `flagged = r for r in records if r['amount'] > 1000` **without the square brackets** — this is actually valid Python (a bare generator expression), but behaves differently from a list: `len()` on it, or iterating it a second time, won’t work the way a list does.

Worth a brief note if a student tries this: the square brackets specifically are what make it a *list* comprehension, materializing all the results at once into a real list.

- Misreading comprehension order — writing the `if` before the `for` (invalid syntax for a filtering comprehension) — a good moment to re-state the fixed order explicitly: `[expression for item in iterable if condition]`, always in that sequence.

Check for understanding: “Rewrite `test_flag_filter`’s comprehension as a full `for` loop with `.append()`, out loud, without looking at the board.” (A good direct check that the comprehension’s compactness hasn’t obscured the underlying, already-familiar accumulator logic — every student should be able to produce the expanded version fluently at this point in the semester.)

6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)

Teaching goal: The now-familiar five-step ritual, chained more compactly than prior weeks — a natural progression as the sequence becomes second nature.

Say to the class:

“Same ritual, written slightly more compactly today — chained with &&, the way you’ll likely start typing it yourself once it’s fully automatic.”

Live-code this:

```
ruff format . && ruff check . && pytest  
git add . && git commit -m 'lab 6: conditionals loops dicts' && git push
```

Line-by-line explanation: Functionally identical to Weeks 4–5’s ritual, just chained with && into two lines instead of five separate ones — say explicitly: **&& chaining means if ruff format . somehow failed, ruff check . and pytest wouldn’t even attempt to run** — a nice, automatic safety property, not just a typing shortcut. Same logic applies to the second line: a failed `git add .` would stop the commit and push from happening.

Common student mistakes to watch for:

- Assuming the chained form is required, or that the five-step spelled-out version from Weeks 4–5 is somehow now wrong — both are equally valid; the chained form is shown here as a natural evolution, not a replacement students must adopt immediately if they’re more comfortable with the explicit version.
- Forgetting “lab 6” in the commit message — the same graded requirement pattern as every prior week; a quick final check before `git push`.

Check for understanding: “If `ruff check .` reported an error, what would you see happen with the chained version above — would `pytest` still attempt to run?” (No — && chaining stops at the first failure; get a student to state this explicitly as the practical benefit of chaining over just running each command on its own separate line, where a failure wouldn’t automatically prevent the next line from being typed and run.)

6.7 Stretch — Sort and Group Output (1:06–1:15, as time allows)

Frame as a quick, genuinely reusable pattern if the room reaches it:

```
sorted_recs = sorted(records, key=lambda r: r['amount'], reverse=True)
print('\nAll records largest to smallest:')
for r in sorted_recs:
    print(f"  ${r['amount']:>8,.2f}  {r['name']} ({r['category']})")
```

Verified output:

All records largest to smallest:

```
$3,500.00  Morgan (Software)
$2,200.00  Alex (Equipment)
$1,200.00  Taylor (Travel)
$  450.00  Jordan (Equipment)
$   89.00  Riley (Travel)
```

Two things worth stating explicitly if you demo this live:

- `sorted(records, key=lambda r: r['amount'], reverse=True)` — `key=lambda r: r['amount']` tells `sorted()` **what to sort by**, since a list of dictionaries has no inherent order on its own — the `lambda` here is a tiny, unnamed function saying “for each record `r`, use `r['amount']` as the sort value.” `reverse=True` flips it to descending. This is genuinely reusable syntax worth naming explicitly: **any time you need to sort a list of dicts by one specific field, this `key=lambda r: r[...]` pattern is the standard tool.**
- `{r['amount']:>8,.2f}` — a new format-spec detail: `>8` right-aligns the value within an 8-character-wide field, so all the dollar amounts line up in a clean column regardless of how many digits each one has — worth a quick comparison of the output with and without the `>8` to make the alignment effect visible.

7 Wrap-Up (last ~9 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including “lab 6”
- ☐ week6_lab.py contains all three parts (records/loop, business rules, summary/file output)
- ☐ data/week6_summary.txt exists and matches the printed summary
- ☐ tests/test_week6.py contains all five tests, all passing
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 7: “Today’s business-rule logic — the LIMIT/HIGH checks, the flagging loop — is exactly the kind of code that’s worth packaging into a reusable function, rather than retyping for every new dataset. Next week: functions, modules, and testing them properly.”

8 Appendix A — Full Answer Key (week6_lab.py + tests/test_week6.py)

```
# week6_lab.py
# Author: [Your Name]

# --- Part 1 ---
records = [
    {'id': 1, 'name': 'Taylor', 'category': 'Travel', 'amount': 1200, 'status': 'Pending'},
    {'id': 2, 'name': 'Jordan', 'category': 'Equipment', 'amount': 450, 'status': 'Pending'},
    {'id': 3, 'name': 'Morgan', 'category': 'Software', 'amount': 3500, 'status': 'Approved'},
    {'id': 4, 'name': 'Riley', 'category': 'Travel', 'amount': 89, 'status': 'Pending'},
    {'id': 5, 'name': 'Alex', 'category': 'Equipment', 'amount': 2200, 'status': 'Pending'}
]

for rec in records:
    if rec['status'] == 'Pending':
        print(f"{rec['name']}: ${rec['amount']:,.2f}")

# --- Part 2 ---
LIMIT = 1000
HIGH = 2000
total = 0
flagged = []
high_value = []

for rec in records:
    if rec['status'] == 'Pending':
```



```

        total += rec['amount']
    if rec['amount'] > LIMIT:
        flagged.append(rec)
    if rec['amount'] > HIGH:
        high_value.append(rec)

# --- Part 3 ---
import os
os.makedirs('data', exist_ok=True)

lines = [
    f'Pending total:    ${total:,.2f}',
    f'Needs review:     {len(flagged)}',
    f'High-value:        {len(high_value)}',
]
for line in lines:
    print(line)

with open('data/week6_summary.txt', 'w') as f:
    f.write('\n'.join(lines) + '\n')

print('\nRecords needing review:')
for r in flagged:
    print(f"   ID {r['id']}: {r['name']} - ${r['amount']:,.2f}")

```

tests/test_week6.py — verified passing as provided:

```

def test_accumulator():
    amounts = [500, 1500, 200]
    total = sum(a for a in amounts)
    assert total == 2200

def test_flag_filter():
    records = [{'amount': 500}, {'amount': 1500}, {'amount': 800}]
    flagged = [r for r in records if r['amount'] > 1000]
    assert len(flagged) == 1

def test_status_filter():
    records = [{'amount': 500, 'status': 'Pending'}, {'amount': 1000, 'status': 'Approved'}]
    pending = [r for r in records if r['status'] == 'Pending']
    assert len(pending) == 1

```

```
def test_dict_key_access():
    rec = {'id': 1, 'amount': 750, 'status': 'Pending'}
    assert rec['amount'] == 750

def test_list_of_dicts_length():
    data = [{'x': 1}, {'x': 2}, {'x': 3}]
    assert len(data) == 3
```

Stretch (sort and group):

```
sorted_recs = sorted(records, key=lambda r: r['amount'], reverse=True)
print('\nAll records largest to smallest:')
for r in sorted_recs:
    print(f"  ${r['amount']:>8,.2f}  {r['name']} ({r['category']})")
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts plus the ritual fill the full class period at a normal pace. If a section moves unusually fast:

Extra — a third business rule. Have students add a `by_category` accumulator — a dictionary mapping each category name to its total pending amount, using the `.get(key, 0)` accumulate-into-a-dict pattern — and print it sorted by category name. (Verified: Equipment: 2650, Travel: 1289 — Software/Morgan is excluded since it's Approved, not Pending.)

Extra — a second comprehension round. Have students rewrite Part 2's entire business-rules block using list comprehensions instead of a `for/if/.append()` loop: `flagged = [r for r in records if r['status'] == 'Pending' and r['amount'] > LIMIT]`. Compare the result against Part 2's original loop-based version — same `flagged` contents, more compact code — a good rehearsal of Part 4's comprehension syntax applied back onto Part 2's real logic, and a natural moment to discuss when a comprehension improves readability versus when a spelled-out loop is still clearer (a comprehension combining several conditions, like this one, can start to feel more cramped than clarifying).